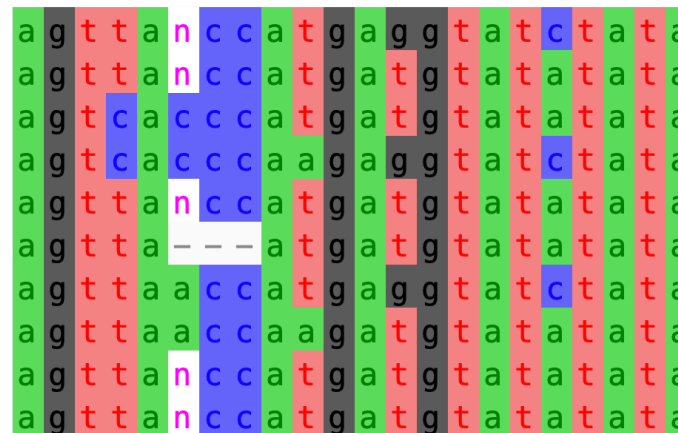




# Variant Detection

*Infectious Disease 'Omics Short Course*



# Outline

## Introduction

- What are the different types of small variants?
- What are the different types of structural (large) variants?

## Variant calling from next generation sequencing data

- What are the major steps of a variant discovery pipeline?
- What are VCFs?

## Discovery of small variants (SNPs and indels)

- What tools are used to detect small variants?
- Variant discovery with GATK and *bcftools*

## Discovery of large variants (structural variants)

- What are the 4 main approaches used to detect large structural variants?
- What tools can we use to detect large structural variants?

## Conclusions

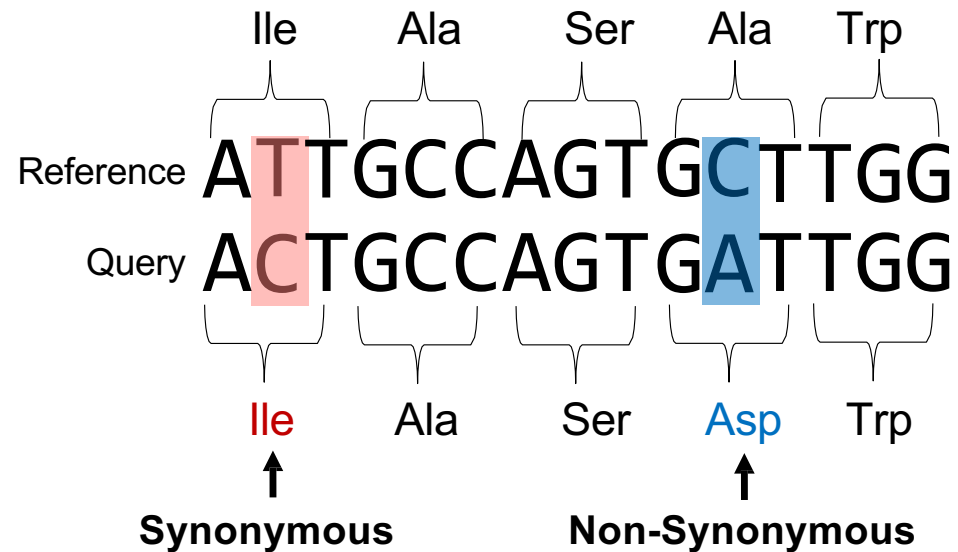
## Practical

# Different types of variants: SNPs

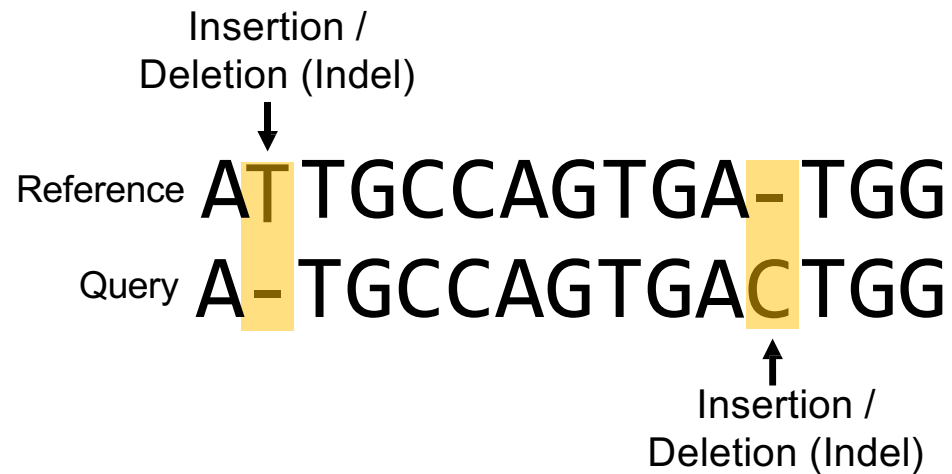


**Single Nucleotide Polymorphisms (SNPs)** are single base pair variations at specific locations in the genome, with respect to a reference genome.

# Different types of variants: SNPs

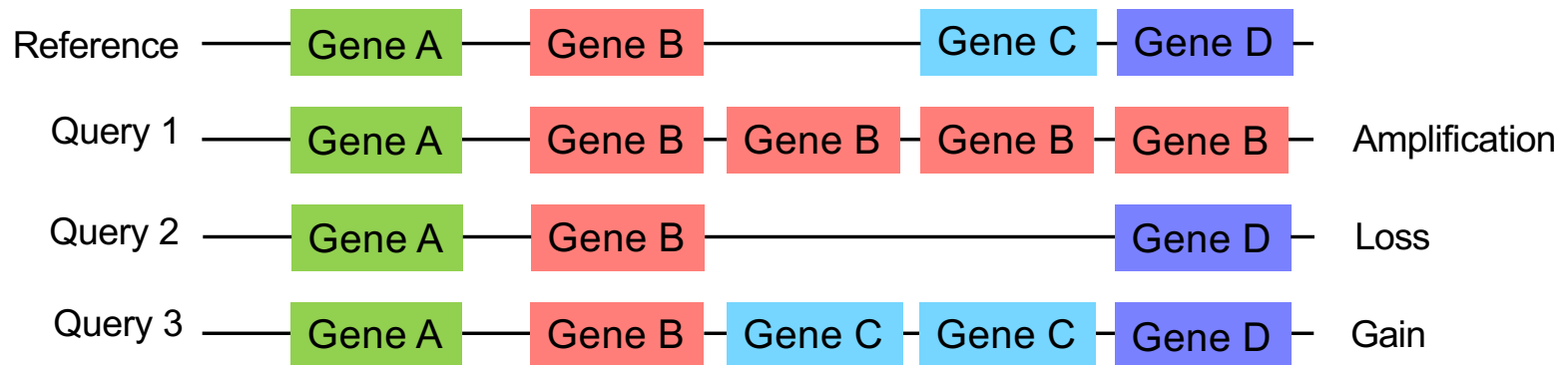


# Different types of variants: Indels



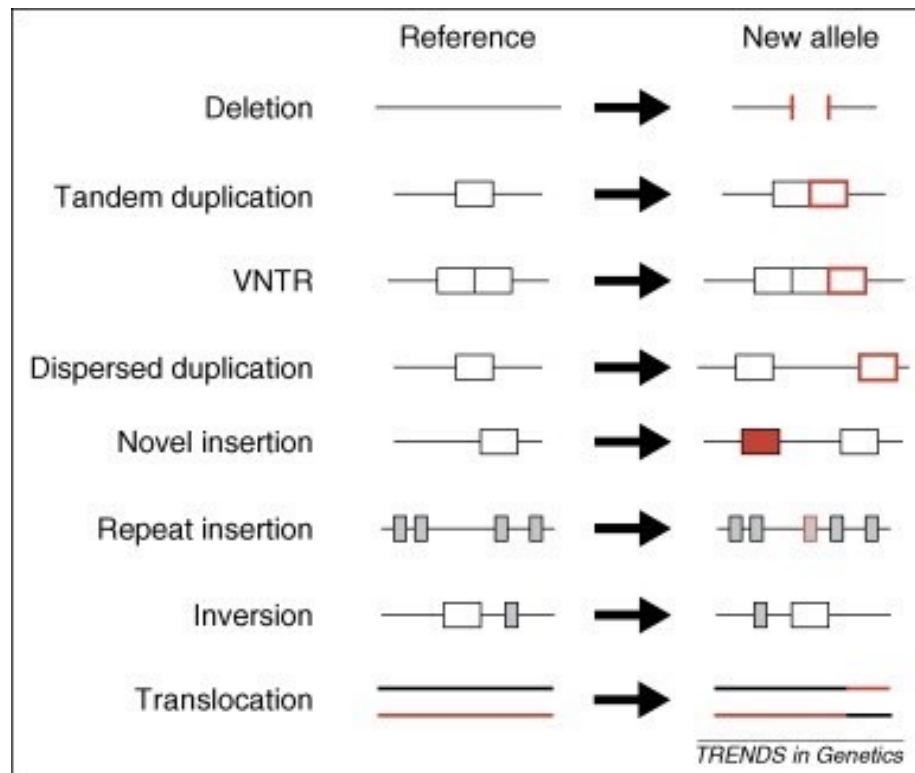
**Insertion-Deletions (Indels)** are insertions and/or deletions of nucleotides at specific locations in the genome, with respect to a reference, and are often events of less than 1kb.

# Different types of variants: CNVs



**Copy Number Variants (CNVs)** are a type of structural variation where the number of copies of a specific segment of DNA varies among different individuals' genomes of the same species.

# Other types of variants



Structural Variants

## • Small scale variants

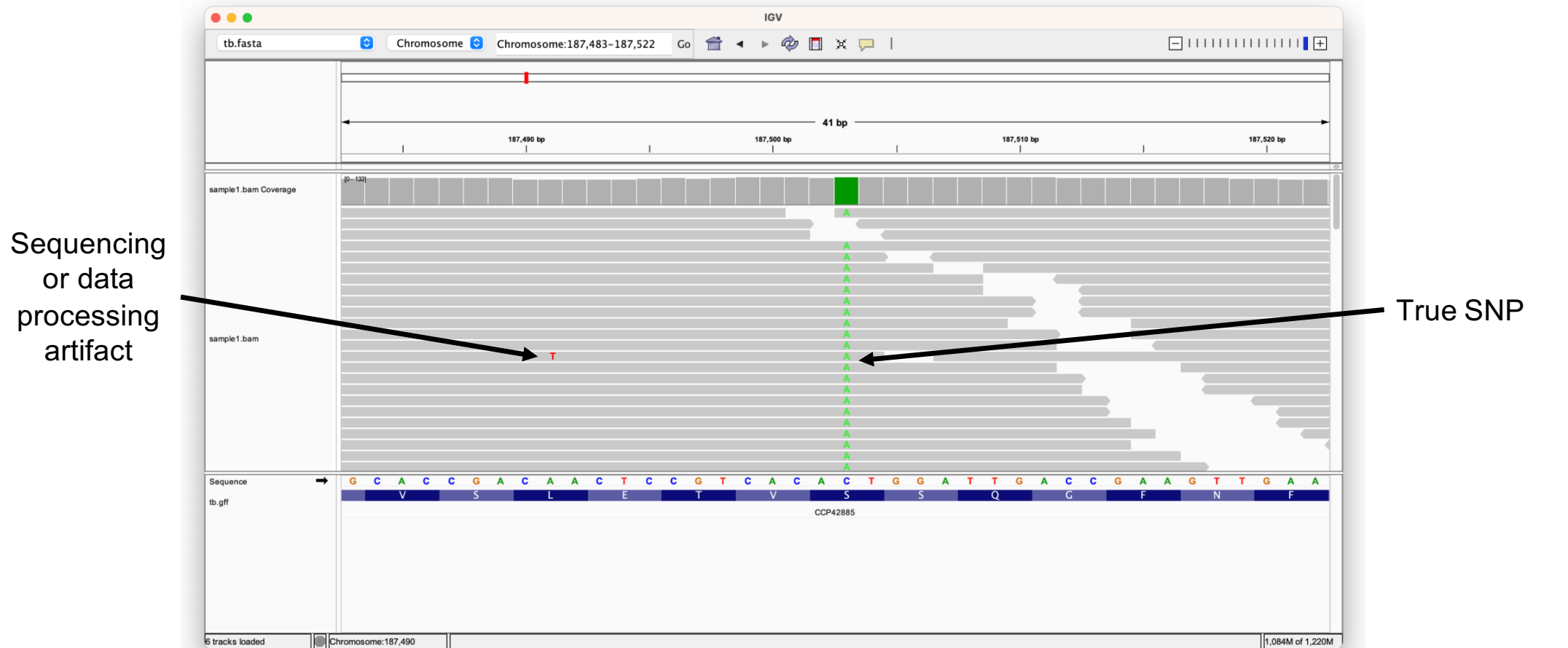
- Single nucleotide polymorphisms (SNPs)
- Insertions and deletions (indels)
- Variable tandem repeats (VNTRs)

## • Large scale variants (>1kb)

- Copy number variations (CNVs)
- Duplications, inversions
- Translocations

...and things in between small and large, and combinations of the above

# Detecting SNPs from alignments

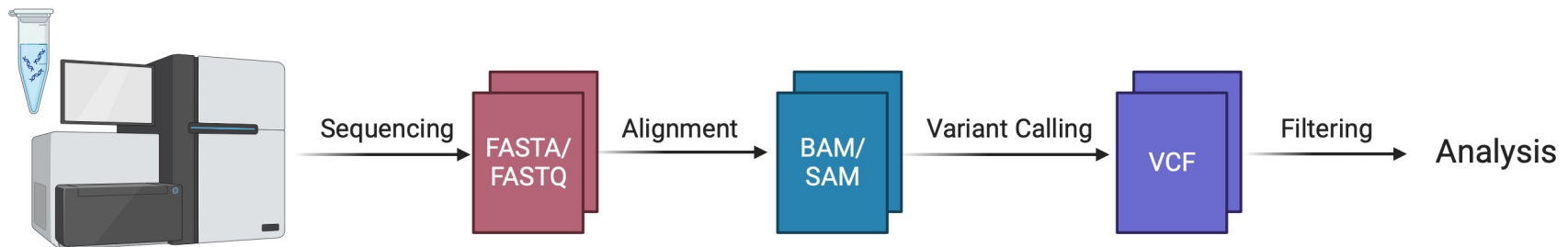


Viewing *Mtb* data in IGV



# Variant Discovery Pipeline for NGS data

- **Aim:**
  - Start with sequencing reads and perform a series of steps to determine the presence of genetic variants
- **Process:**
  - Creation of the variant call format (VCF) file...



# Variant Call Format (VCF)

- A VCF is a text file format employed to store genetic variation with respect to a reference genome.

```
##fileformat=VCFv4.1
##fileDate=20110413
##source=VCFtools
##reference=file:///refs/human_NCBI36.fasta
##contig=<ID=1,length=249250621,md5=1b22b98cdeb4a9304cb5d48026a85128,species="Homo Sapiens">
##contig=<ID=X,length=155270560,md5=7e0e2e580297b7764e31dbc80c2540dd,species="Homo Sapiens">
##INFO=<ID=AA,Number=1,Type=String,Description="Ancestral Allele">
##INFO=<ID=H2,Number=0,Type=Flag,Description="HapMap2 membership">
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=GQ,Number=1,Type=Integer,Description="Genotype Quality">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##ALT=<ID=DEL,Description="Deletion">
##INFO=<ID=SVTYPE,Number=1,Type=String,Description="Type of structural variant">
##INFO=<ID=END,Number=1,Type=Integer,Description="End position of the variant">
#CHROM POS ID REF ALT QUAL FILTER INFO FORMAT SAMPLE1 SAMPLE2
1 1 . ACG A,AT 40 PASS . GT:DP 1/1:13 2/2:29
1 2 . C T,CT . PASS H2;AA=T GT 0|1 2/2
1 5 rs12 A G 67 PASS . GT:DP 1|0:16 2/2:20
X 100 . T <DEL> . PASS SVTYPE=DEL;END=299 GT:GQ:DP 1:12:. 0/0:20:36
```

# Variant Call Format (VCF)

**Header**

```
##fileformat=VCFv4.1
##fileDate=20110413
##source=VCFtools
##reference=file:///refs/human_NCBI36.fasta
##contig=<ID=1,length=249250621,md5=1b22b98cdeb4a9304cb5d48026a85128,species="Homo Sapiens">
##contig=<ID=X,length=155270560,md5=7e0e2e580297b7764e31dbc80c2540dd,species="Homo Sapiens">
##INFO=<ID=AA,Number=1,Type=String,Description="Ancestral Allele">
##INFO=<ID=H2,Number=0,Type=Flag,Description="HapMap2 membership">
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=GQ,Number=1,Type=Integer,Description="Genotype Quality">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##ALT=<ID=DEL,Description="Deletion">
##INFO=<ID=SVTYPE,Number=1,Type=String,Description="Type of structural variant">
##INFO=<ID=END,Number=1,Type=Integer,Description="End position of the variant">
```

**Body**

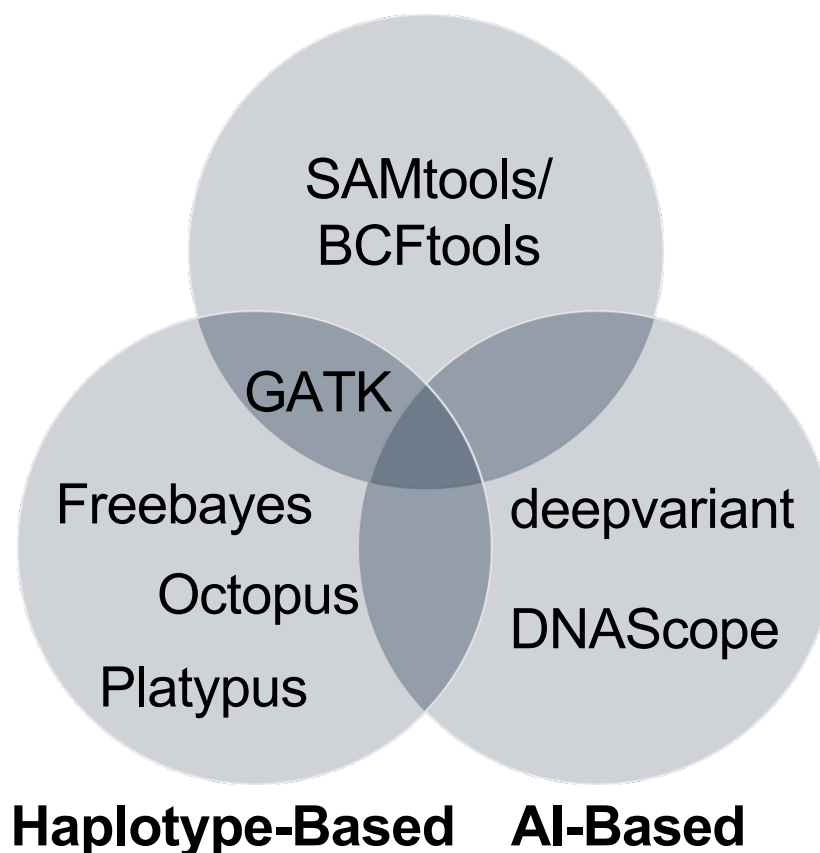
| #CHROM | POS | ID   | REF | ALT   | QUAL | FILTER | INFO               | FORMAT   | SAMPLE1 | SAMPLE2   |
|--------|-----|------|-----|-------|------|--------|--------------------|----------|---------|-----------|
| 1      | 1   | .    | ACG | A,AT  | 40   | PASS   | .                  | GT:DP    | 1/1:13  | 2/2:29    |
| 1      | 2   | .    | C   | T,CT  | .    | PASS   | H2;AA=T            | GT       | 0 1     | 2/2       |
| 1      | 5   | rs12 | A   | G     | 67   | PASS   | .                  | GT:DP    | 1 0:16  | 2/2:20    |
| X      | 100 | .    | T   | <DEL> | .    | PASS   | SVTYPE=DEL;END=299 | GT:GQ:DP | 1:12:.  | 0/0:20:36 |

**Annotations:**

- SNP:** Points to the variant at position 5 (rs12).
- Large SV (Deletion):** Points to the deletion variant at position 100.
- Insertion:** Points to the insertion variant at position 2 (CT).
- Phased Genotype Data:** Points to the 0|1 genotype in Sample 1.

# SNP and short indel calling tools

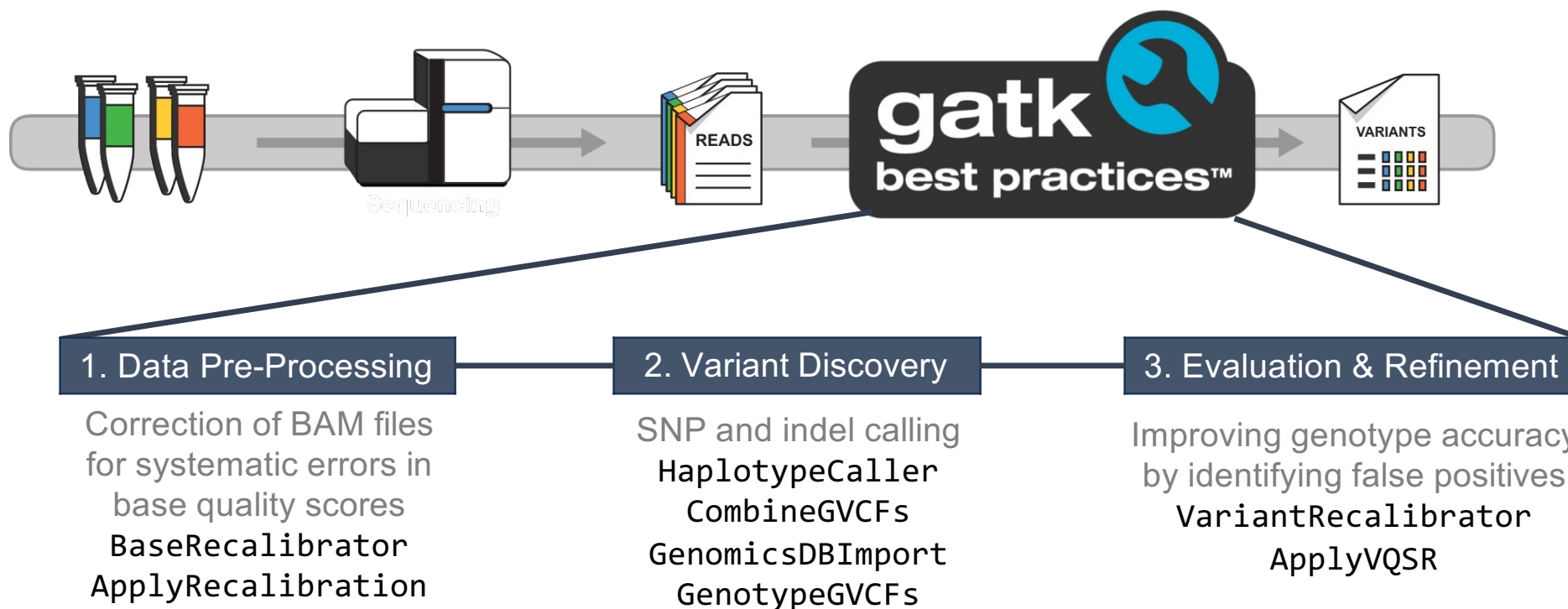
## Base Callers



All (described here) are used by the open-source community.

All are performing variant calling but **employing different models to do so.**

# Variant discovery with GATK



GATK is a package of command-line tools written in Java and provides end-to-end workflows called best practices. It is easily parallelised and scalable, but run times are long!

# Variant discovery with bcftools

A two-stage process using two algorithms:

mpileup

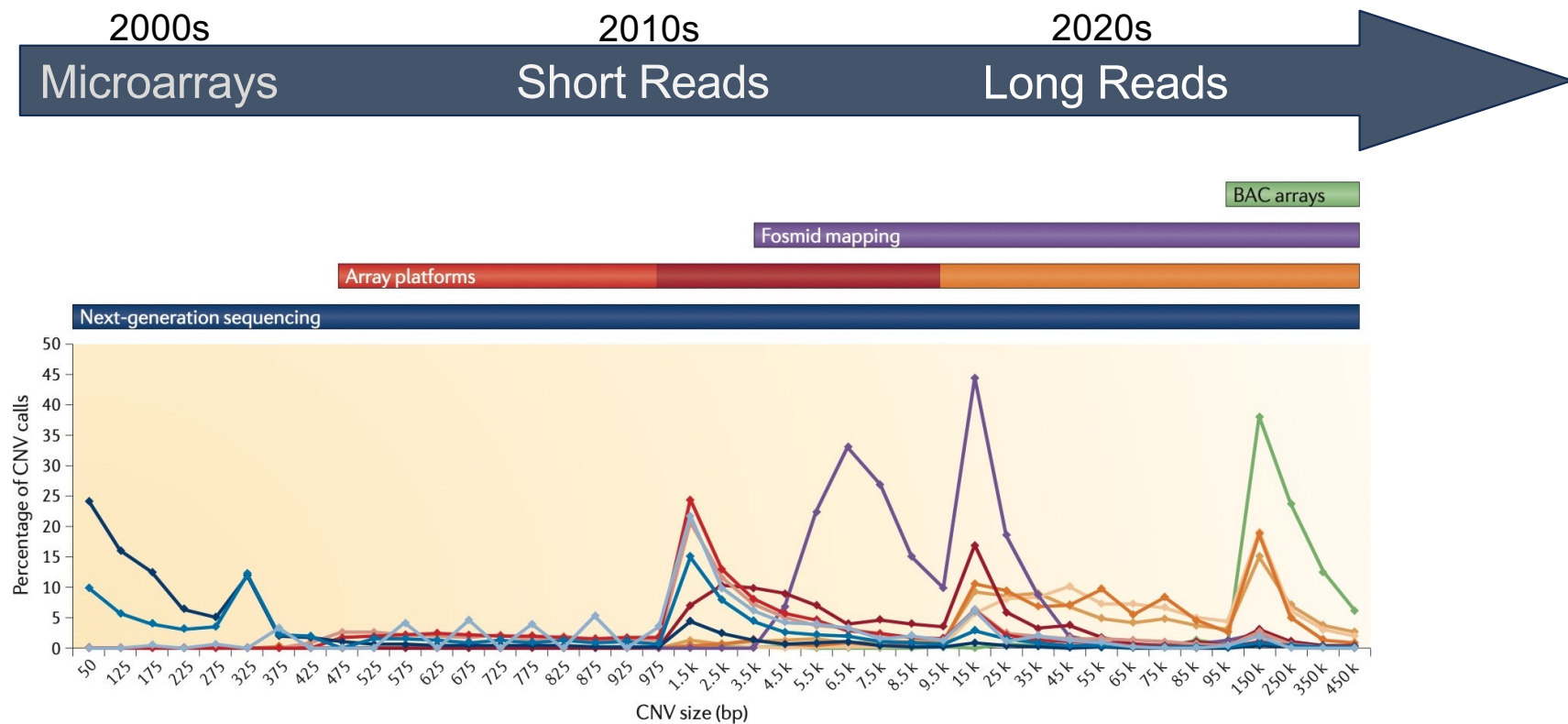
- Aggregation of all reads at each position
- Calculation of genotype likelihoods from a BAM file

call

- Selects most likely genotype and compares with the reference genome, outputting a VCF

Much **faster** than GATK!

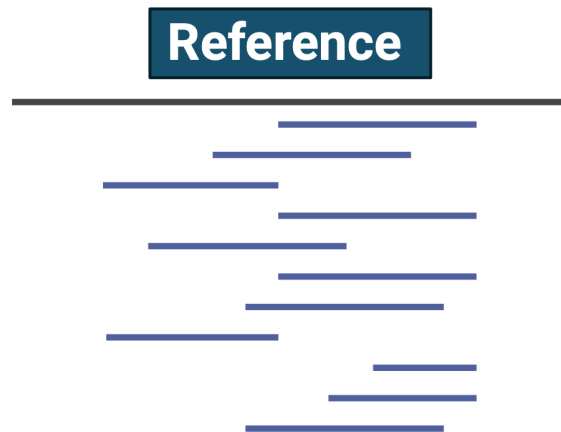
# Next generation sequencing and SVs



Next generation sequencing offers the widest range of detection of structural variants.

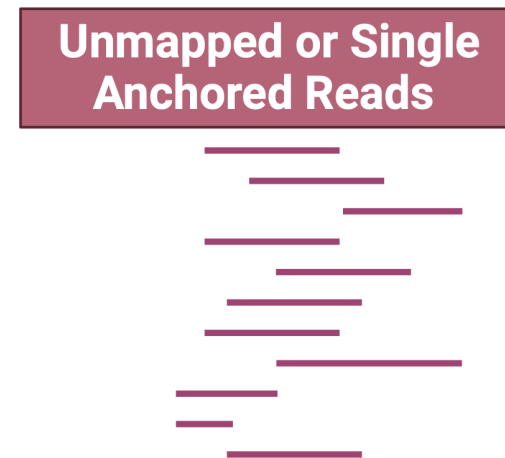
# Discovery of structural variants

When short read data are mapped to a reference, structural variants can be identified by their unique signatures.



## Approaches:

Paired End, Split Read, Read Depth



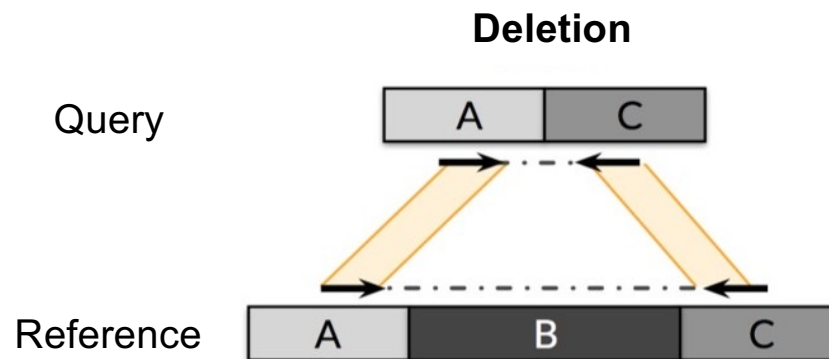
## Approaches:

Assembly (Discussed on Day 2)

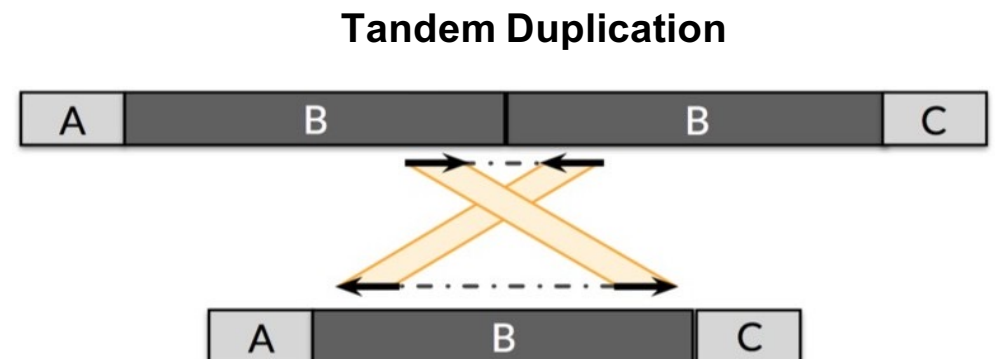


# Paired End (PE) Approach

- Assesses the span and orientation of PE reads.
- If an SV is present, it will produce 'discordant' alignments.



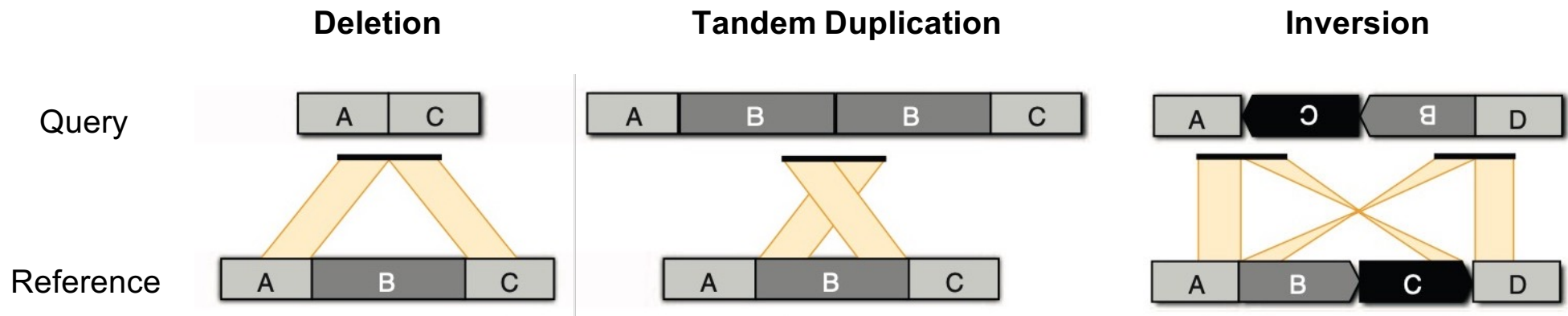
Reads mapping further apart than expected with respect to reference



Reads mapping in the opposite orientation than expected with respect to the reference

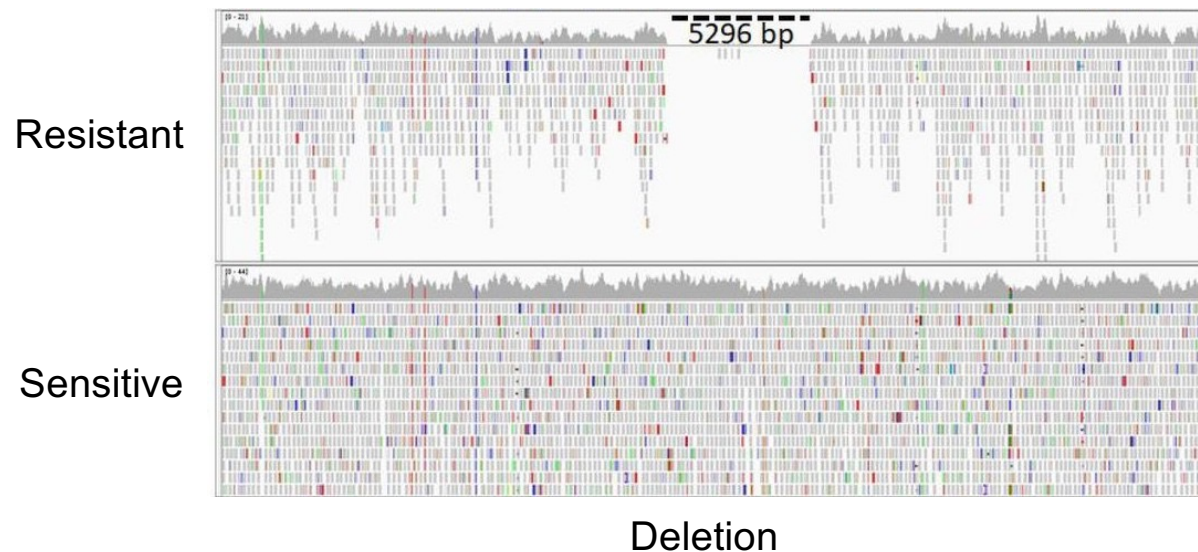
# Split Read (SR) Approach

- Identifies sequences containing a breakpoint, mapping them to single base-pair resolution.



# Read Depth (RD) Approach

- Detects deletions or duplications based on divergences in mapping depth:
  - Low or zero coverage suggests a deletion
  - Excess coverage suggests a duplication



# Summary of SV detection and tools

| SV classes               | Read pair | Read depth     | Split read | Assembly |
|--------------------------|-----------|----------------|------------|----------|
| Deletion                 |           |                |            |          |
| Novel sequence insertion |           | Not applicable |            |          |
| Mobile-element insertion |           | Not applicable |            |          |
| Inversion                |           | Not applicable |            |          |
| Interspersed duplication |           |                |            |          |
| Tandem duplication       |           |                |            |          |

Low

Resolution

High

RD

PE

SR

Assembly

| CNV calling tool     | Method | Citations | Year |
|----------------------|--------|-----------|------|
| VarScan2             |        |           | 2016 |
| Pindel               |        |           | 2015 |
| BreakDancer          |        |           | 2010 |
| <b>CNVnator</b>      |        |           | 2019 |
| mrCaNaVar            |        |           | 2013 |
| <b>DELLY</b>         |        |           | 2019 |
| RDXplorer            |        |           | 2011 |
| SegSeq               |        |           | 2008 |
| CNV-seq              |        |           | 2009 |
| <b>LUMPY</b>         |        |           | 2017 |
| CREST                |        |           | 2011 |
| CONIFER              |        |           | 2011 |
| ExomeCNV             |        |           | 2012 |
| <b>Control-FREEC</b> |        |           | 2018 |
| XHMM                 |        |           | 2012 |
| <b>ExomeDepth</b>    |        |           | 2016 |
| <b>cn.MOPS</b>       |        |           | 2014 |
| Hydra                |        |           | 2010 |
| PEMER                |        |           | 2008 |
| CONTRA               |        |           | 2016 |
| <b>Manta</b>         |        |           | 2018 |
| SVDetect             |        |           | 2013 |
| <b>CNVkit</b>        |        |           | 2016 |
| CNVr                 |        |           | 2010 |
| ReadDepth            |        |           | 2011 |
| GASVPro              |        |           | 2011 |
| <b>CLC</b>           |        |           | 2019 |

...Again, all are essentially performing structural variant calling but employing different methods to do so.

# Conclusions

- Many different types and combinations of variants
- A VCF stores data on variants:
  - It is the output for several variant calling software (e.g., GATK)
  - It is the input for downstream filtering and analysis (e.g., population genetics)
- Detection of small vs. large variants requires different approaches.
  - Whichever strategy employed comes with its own advantages and disadvantages.
- **It is a combination of the choice of software tools for both alignment and variant calling that will influence the final result (i.e., variants called).**
  - Use whatever works best for your research question/project!

a g t t a n c c a t g g t a t c t a t a  
a g t t a n c c a t g a t g t a t a t a t a  
a g t c a c c c a t g a t g t a t a t a t a  
a g t c a c c c a a g a g g t a t c t a t a  
a g t t a n c c a t g a t g t a t a t a t a  
a g t t a - - - a t g a t g t a t a t a t a  
a g t t a a c c a t g a g g t a t c t a t a  
a g t t a a c c a a g a t g t a t a t a t a  
a g t t a n c c a t g a t g t a t a t a t a  
a g t t a n c c a t g a t g t a t a t a t a

# Practical